-- LaimisJ - 02 Feb 2005

- Network topology
  - Bus Termination
  - The Cable
- Protocol description
  - Module ID (Address)
  - Register ID
  - Message address field
  - Address groups
    - Priority
    - Destination.
    - Source.
    - Code.
    - Reg ID.
  - Data field.
  - Data display format.
- Common procedures
  - Trigger read register value request
  - Write register value
  - Discovery of online devices
  - Device ID programing
  - Error detecting
  - Menu file
    - Getting menu file
    - Menu file structure
  - Error file
    - Getting Error file
    - Error file structure
    - Error extension 2021
  - Emergency stop
  - 2019 protocol changes

Network topology

- Physical layer defined by the CAN standard, part ISO 11898-2, is used. It uses a two-wire balanced signaling scheme.
- Bus speed fixed at 1 Mbit/s

Bus Termination

- An ISO 11898 CAN bus must be terminated. This is done using a resistor of 120 Ohms in each end of the bus. The termination serves two purposes:
    1. Remove the signal reflections at the end of the bus.
    2. Ensure the bus gets correct DC levels.

The Cable

- The ISO 11898 prescribes that the cable impedance be nominally 120 Ohms, but an impedance in the interval of [108..132] Ohms is permitted.
- Maximum Cable Length. At a speed of 1 Mbit/s, a maximum cable length of about 40 meters (130 ft.) can be used. This is because the arbitration scheme requires that the wave front of the signal can propagate to the most remote node and back again before the bit is sampled.
- It is necessary to use shielded cables if the CAN bus extension exceeds 3 meters.
- Node count is limited to 62.
- PS5012CANNET_ASS.pdf: PS5012 network connection schematics
- The pin assignment for the DB-9 'CAN' plug and socket is shown in the table below

DB-9 Pin	GSignal
2	CAN-L
3	GND
7	CAN-H

Protocol description

- This means that the data is transmitted on the network with directing to a specific destination.
- There is no pure slave and master devices on the bus. Slave devices, by convention, passively sit and wait for a remote master device to ask them to report existing data values (Read) or accept new data values (Write).
- Most of modules are in slave mode after power on, but may be programmed to report data and status on event or periodic basis.
- Protocol does not have module announce nor heartbeat message specified, therefore cyclic device discovery procedure should by used for live modules

detection.
- Even dedicated bus master module, e.g. master controller board will act as slave if other device will ask report data.

Module ID (Address)

- Module ID is device address as it appears on the bus and at the device interface. Module ID can be any number from 1 to 62 (1..3E HEX). ID=63 is reserved for PC, ID=0 for broadcast. For a given network, every address must be unique. This means that two different devices cannot use the same ID. In general, most addresses are preassigned and used immediately upon power up. A specific procedure for assigning addresses is available if required. In case several modules of the same type you are going to connect to the bus, please check ID in advance.
- Each device on the network will have at least one name and one address associated with it. There are examples, such as several different laser modules residing in a common "device", wherein multiple device names and multiple addresses may coexist within a single electronics unit.

Register ID

- Register ID is a separately addressable entity within a module. A module contains at least one and no more than 255 registers. Register ID from 0 to 15 are reserved for protocol implementation.
- A register represents a single operational or calibration parameter.
- There are several basic types of registers available:
  - Numerical. Includes integer and floating types.
  - Enumerated. Gives a set of actions or statuses and index byte from which to select.
  - String - byte array.

Message address field

- CAN 2.0B (29-bit identifiers) is used
- For bit description see table below

Bit	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	0
Group	Priority		Destination						Code							Source						Reg ID							

- Data frame and remote frame are used.
- Remote frame (RTR byte is set) indicates request for register values.
- (RTR = 0) Data frames are used for sending and receiving information

Address groups

Priority

- The first 2 bits are used for determining message priority during the arbitration process. A value of 00 has the highest priority. Higher priority messages would typically be used for high speed control messages.
- Standard priority is used = 3

Destination.

- Address of the receiver module. Message with 'Destination'=0 are intended to be broadcast and it accepts all active modules.
- See **ID column** from *.csv file for IDs.
- Range 1..$3F

Source.

- Sender ID. When module answers to request source and destination IDs are exchanged.
- Modules have ID range 1..$3E . ID = $3F reserved for PC

Code.

- **code** bits identifies how data should be processed after message is received.

Message ID bit	20	19	18	17	16	15	14
Value	cNS,cRESET	cNV	cREQ	cWR	cnF		

- cNS turns on reporting of register value updates (subscribing)
- cRESET resets file transfer
- cNV makes register value nonvolatile
- cREQ requests updated register value in case cWR is set. Don't care when cWR is not set.
- cWR write register value
- cnF do not forward. All devices should keep it 0, bridges set it to allow access to bridge registers behind the wall.
- **Code** examples:
    - Write and request value: Code:=cWR+cREQ;
    - Write, make nonvolatile and request value: Code:=cWR+cNV+cREQ

- Read register value: code:=cREQ, RTR, DLC=0;
- Read register value and subscribe for updates: Code:=cNS+cREQ, RTR, DLC=0

Reg ID.

- Registers keeps single operational or control parameter. They are addressed by ID.
- For ID see **Register ID** column from *.csv file
- Regular register IDs are between $10..$7F
- ID range 0..15 is reserved for control registers:

RegID	Description	Type
0	Device Name	str8
1	Menu file	file
2	Error code	u16
3	Error string	ASCIIZ
4	Error file	file
5	Device ID	u8,NV
6	CAN SYNC	DLC=0, RTR
7	File	file
8	Emergency stop	u32
9	Firmware version	u32
10	Network features. bit0 gateway. bit1:4 CANFD speed	u32
11	Far forward register. Allows access through two bridges	

Data field.

- See **Type** column from *.csv file for register type
- Data length (DLC) depends on register type, see table below.

Type	Description	DLC	RegType
u8	byte 0..255 unsigned 8-bit	1	0
s8	shortint -128..127 signed 8-bit	1	1
u16	word 0..65535 unsigned 16-bit	2	2
s16	smallint -32768..32767 signed 16-bit	2	3
u32	longword 0..4294967295 unsigned 32-bit	4	4
s32	long integer -2147483648..2147483647 signed 32-bit	4	5
float	IEEE754 single precision +-3.4 x 10^38 Significant digits 6-7	4	6
string8	array of bytes	8	7
u64	C unsigned long long 0..2^64-1 unsigned 64-bit	8	8
s64	C signed long long word -2^63..2^63-1 signed 64-bit	8	9
double	IEEE754 double precission signed long long word +-1.7 x 10^308	8	10

Data display format.

- Data display format helps with numerical value interpretation.
- See **Print format** column from *.csv file
- Integer types **%u[x]** There x is parameter dimension.
- Decimal types. Register value is integer. Print format e.g. **%.1f** indicated that register value should be divided by 10 for display, **%.2f[x]** - by 100...
- Float type **%f[x]**
- Set type. Register type is u8. Register value points to corresponding actual parameter from the set.

Common procedures

Trigger read register value request

- Remote frame (RTR set to 1) is sent to request register value:
  - **Priority** = any value
  - **Destination** = device ID
  - **Source** = any value ($3F default value for PC)

    - **Code** = cREQ or **Code** = cNS+cREQ
    - **Reg ID**
    - **DLC** = 0
    - **Data** = none
- If **Code** has cNS bit set as well, module answers with current register value, also subscribes for register updates. In this mode module reports about register updates periodically or on register value updates. As destination, ID of requesting source module is used. Please note:
    - only read only registers have this feature
    - how often report is sent, depends on data acquisition mode: it may be periodic, synchronic or event based.
    - register report mode may be reset by sending remote frame with cNS bit negated in **Code**.

Write register value

- Data frame:
    - **Priority** = any value
    - **Destination** = device ID
    - **Source** = any value ($3F default value for PC)
    - **Code** = cWR or any combination with cNV, cREQ
    - **Reg ID**
    - **DLC** = should match register type size.

- There is no direct confirmation about successful writing. This is because usually modules will coerce to the nearest limit if parameter is out of bounds. For example, let's x register have available range 0..10. If one will send write command with parameter value 11, actual value will be set to 10. If one cares about write command results, should use Code:=cWR+cREQ, then module responds with data frame containing actual new register value.

Discovery of online devices

- broadcast message requesting device name register value is used for device discovery.
- Send remote frame (RTR set to 1) to CAN to scan on-line modules:
    - **Priority** = any value
    - **Destination** = 0
    - **Code** = cREQ
    - **Source** = any value ($3F default value for PC)
    - **Reg ID** = 0 (device name register)
    - **DLC** = 0

- As alternative, packet with destination from 1 to $3E may be sent to CAN. No reply will mean device with given ID is not present.

Device ID programing

- Usually modules are programmed to the default IDs. If there is a several modules of the same type on-line, IDs should be changed in advance in order to avoid collision. It is impossible to download menu file in case there is ID recurrence. Also recurrence can't be solved unless modules with duplicating IDs are disconnected for network.
- Device ID is programmed by standard register altering procedure.
- Following message is sent to CAN to change module ID:
  - **Priority** = any value
  - **Destination** = current device ID
  - **Code** =cWR+cNV = $08 + $20 = $28
  - **Source** = any value ($3F default value for PC)
  - **Reg ID** = 5
  - **DLC** = 1
  - **Data** = new device ID, value 1..$3E
- Notes
  - Destination = 0 may be used. Then all modules on-line gets new ID.
  - Module power should be cycled in order to validate new ID.
  - No confirmation is sent from device side. After power cycle use device discovery procedure in order to check programming success.

Error detecting

- There is a dedicated register ID =2
- Register may be polled for the module status
- Value = 0 indicates faultless operation
- Value <> 0 means ,that the error was triggered
- Register may be requested with remote frame (RTR=1), **Code** = cNS+cREQ and report mode will be turned on. In this case Module will send data frame with register value if error condition occurs.
- Protocol do not describes how error is cleared. This depends on module and error type, see product manual. Reading of register does not clear error flag.

Menu file

- Menu file contains information about internal registers structure. It is used by CAN browser to manage module information as tree of registers for viewing and

altering. This useful in dynamic systems, because information about internal registers structure may be downloaded directly from module in several seconds.
- For static systems once downloaded file may be used instead of.

Getting menu file

- Menu file register (register ID = 1) and Errors file register (register ID = 4) are special purpose registers. These registers allow to transfer from device a binary file. Binary file transfer protocol is as follows.
    1. Host device initiates binary file transfer by sending message with cRESET,cREQ and RTR bits set. cRESET resets file pointer to the start of file. [cRESET=1, cNV=0, cREQ=1, cWR=0, RTR=1, DLC=0]
    2. Device resets file pointer if cRESET is set. Device copies up to 8 octets from binary file to message buffer. Device copies less than 8 octets when file pointer reaches the end of file. Device advances file pointer by 8 or to the end of file and sends message to the host with following attributes: [cRESET=0, cNV=0, cREQ=1, cWR=0, RTR=0, DLC=0.
    3. Host checks if the length of data payload of received message is 8 or less. If it is 8, then host handshakes file reception by sending message like in step 1, but with cRESET=0 [cRESET=0, cNV=0, cREQ=0, cWR=0, RTR=1, NO DATA]. If the length of data payload of received message is less than 8, then file transfer is complete.
    4. Steps 2 and 3 are repeated until device replies with length of data payload less than 8.

Menu file structure

- Menu file is binary file. It consists of header and body. [Header][Body]
- Header consists of two octets representing CRC-16-CCITT checksum of menu file body,low endian format. Header=[ChecksumLO][ChecksumHI]
- Menu file Body consists of variable length records Body=[TreeRec][TreeRec]...
- Where TreeRec is menu tree node descriptor. It is a binary, variable length record consisting of following fields:
    - [RegId]
    - [TreeId]
    - [RegType]
    - [Permissions]
    - [Low limit]
    - [High limit]
    - [Short name]
    - [Format]
    - [Long name]
- **RegId** is a fixed length field, single octet, representing the register ID.
- **TreeId** is fixed length field, single octet, representing menu tree id code. Upper four bits represent menu child node number of the tree and lower four bits represent the position index in the child node. TreeId=[TreeNode:TreeNodeIdx]

- **RegType** is a fixed length field, single octet, representing register type.
- **Permissions** is a fixed length field, single octet, representing permissions bits.

Bit	7	6	5..4	3..2	1..0
Function	-	NV	Advanced mode	User mode	Service mode

- ◦ Decoding for Advanced, Service, User modes is following : 0 - not visible, 2 - read only, 3 - read/write
  ◦ Permissions is used to limit access to a particular register in external software, e.g. CAN browser. There is no access limiting in CAN message level. External Software publisher is responsible for granting proper rights to user groups.
  ◦ Altering of registers with only advanced mode access should by avoided in any case, except manufacturing testing and calibrating procedures.
  ◦ NV - indicates that register may be written to the nonvolatile memory.
- **Low limit** is a variable length field representing the bottom limit of register value. Field length is 1 octet for 8-bit registers, 2 octets for 16-bit registers, 4 octets for 32-bit registers. Byte order is low endian
- **High limit** is variable length field representing the top limit of register value. Field length is 1 octet for 8-bit registers, 2 octets for 16-bit registers, 4 octets for 32-bit registers. Byte order is low endian.
- **Short name** is a variable length field representing short register name. Format of Short name field is a zero terminated ASCII string. 'Short name' is intended for using in interfaces with limited screen resolution, e.g. control pad.
- **Format** is a variable length field representing how register value should be formatted, C-like format string. Format of this field is zero terminated ASCII string.
- **Long name** is variable length filed representing long register name. It's a zero terminated ASCII string. Parameter name in descriptive form with unlimited length.

- For example menu tree record TreeRec could look as follows:

Code	Descrition
11h	RegId = 17
01h	TreeId = 1
03h	RegType = signed 16-bits integer
2Ah	volatile, ADV RO, USR RO, SVC RO
70FEh	Low limit = -400
E204h	High limit = 1250
54656D7000h	'Temp' string
252E31664300h	'%.1fC' string
44726976657220746 56D706572617475726500h	'Driver temperature' string

- Note **High Limit** and **Low Limit** are statically defined limits. Actual limits may change during runtime, see note in **Write register value** chapter.

Error file

- Error file lists available error codes and description strings. Module signals error by sending data frame with error code. Readable error explanation may be found from error file by finding the code.

Getting Error file

- Use the same procedure for reading error file as with menu file. Only register ID = 4 should by used instead of ID = 1.

Error file structure

- Error file is a binary file. It consists of **header** and a **body** . [Header][Body]
- **Header** consists of two octets representing CRC-16-CCITT checksum of error file body,low endian format. Header=[ChecksumLO][ChecksumHI]
- Error file **body** consists of variable length records so that Body=[ErrRec][ErrRec]...
- **Errrec** is a variable length record with the following fields:
  - [ErrorBitMask]
  - [ErrorBitNameShort]
  - [ErrorBitNameLong]
- **ErrorBitMask** is a fixed length field, two octets, low endian byte order, representing the bitmask of the error.
- **ErrorBitNameShort** is a variable length field representing short name of error. Format is zero terminated string.
- **ErrorBitNameLong** is a variable length field representing long error name. Format is zero terminated string.
- For example ErrRec record could look as follows

Code	Descrition
0001h	ErrorBitMask = 1
506F77657244776E00h	ErrorBitNameShort = "PowerDwn"
506F776572446F776E00	ErrorBitNameLong = "PowerDown"

Error extension 2021

The old style error file can describe up to 16 errors because it uses a 16-bit error bit mask. The error file of a new style can describe up to 64 errors. Instead of the 16-bit error bit mask in the error file, the encoded error bit number (BitNumber + 0x8001) can be written. BitNumber - 0..63.

Emergency stop

Any device on the bus can cause emergency stop of laser system by writing to device devID=0 (broadcast address), register regID=8. Value, device writes to emergency stop register, should contain in lower 16bits device internal error code. Upper 16bits of emergency stop register are reserved.

2019 protocol changes

- Two times more registers per device. Previously reserved bit 7 now is used in register field.
- Since coming CAN FD standard abandons RTR bit usage, for better new coming devices compatibility all new software while requesting register value or CAN menu should not only set RTR bit (which will be not available in CAN FD networks), but also set cREQ control bit. This will maintain better compatibility and ease upgrade to CAN FD
    - Old cREQ bit description was this:
        - cREQ requests updated register value in case cWR is set. Don't care when cWR is not set.
    - New cREQ description
        - cREQ=1 requests new register value. Used in combination with cWR=1 requests updated register value after write.
    - Old read register value and read register value and subscribe for updates code examples:
        - Read register value: code:=0, RTR;
        - Read register value and subscribe for updates: Code:=cNS, RTR
    - New read register value and read register value and subscribe for updates code examples:
        - Read register value: code:=cREQ, RTR;
        - Read register value and subscribe for updates: Code:=cNS+cREQ, RTR

Topic revision: r36 - 2022-09-28 - LaimisJ

Copyright © 2008-2022 by the contributing authors. All material on this collaboration platform is the property of the contributing authors.
Ideas, requests, problems regarding TWiki? Send feedback